Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Северо-Восточный федеральный университет
имени М.К. Аммосова»
Институт математики и информатики
Кафедра вычислительных технологий

ОПТИМИЗАЦИЯ ФОРМЫ БОКАЛА SPH-МЕТОДОМ

КУРСОВОЙ ПРОЕКТ

Направление подготовки: 01.03.02 - Прикладная математика и информатика

Направленность: Математическое моделирование и вычислительная математика

Руководитель:

К.ф.-м.н., доцент кафедры вычислительных технологий
Григорьев Василий Васильевич

Выполнила:

студент 3 курса группы Б-ПМИ-22
Шарина Валентина Николаевна

Работа защищена:

_____2025 г.

протокол №_____

с оценкой_____

Якутск, 2025

Содержание

Введение 3

Глава 1. Теоретическая часть 4

1.1. Формулировка SPH-метода 4

1.2. Применение SPH-метода в моделировании жидкостей 7

Глава 2. Практическая часть 9

2.1. Разработка моделей бокалов 9

2.2. Численное моделирование наливания жидкости 9

2.3. Анализ результатов 10

Заключение 14

Список литературы 15

Приложения 1 16

Приложение 2 19

Приложение 3 25

Введение

Оптимизация конструкций с учетом динамических нагрузок является важной задачей в машиностроении, аэрокосмической и энергетической отраслях. Традиционные методы (например, сеточный метод и метод конечных элементов) могут быть недостаточно точными при моделировании жидкостно-структурного взаимодействия. SPH-метод (Smoothed Particle Hydrodynamics) позволяет эффективно анализировать такие процессы благодаря бессеточному подходу, повышает точность расчётов и расширяет область применимости численного метода.

Актуальность исследования. Использование численного моделирования позволяет провести анализ поведения жидкости в бокале и подобрать такую форму, которая будет обеспечивать наилучшие характеристики для минимизации расплескивания.

Объект исследования. Оптимизация формы бокала с использованием SPH-метода для снижения брызг.

Предмет исследования. Поведение жидкости при наливании в бокалы разных форм, моделируемое с помощью SPH-метода.

Цель исследования: Разработка оптимальной формы стакана для минимизации брызг с использованием SPH-метода.

Задачи исследования:

1. Изучение SPH-метода.
2. Создание различных форм бокала.
3. Моделирование наливания жидкости (SPH-частицы) для разных форм бокала.
4. Выявить геометрические параметры с минимальным количеством брызгов.

Структура курсового проекта состоит из введения, 2 глав, заключения, списка используемой литературы и приложений.

Глава 1. Теоретическая часть

1.1. Формулировка SPH-метода

Метод сглаженных частиц (SPH, Smoothed Particle Hydrodynamics) – это численный метод, применяемый для моделирования жидкостей и газов, основанный на лагранжевом подходе, в котором непрерывная среда представляется в виде дискретного множества частиц. Каждая частица несёт физические характеристики вещества: массу, плотность, скорость, давление и так далее. Эти частицы взаимодействуют друг с другом на основе локальных вычислений в пределах заданного радиуса влияния.

Для получения такого представления необходимо от непрерывной аппроксимации функции, полученной с помощью сглаживающего ядра, перейти к аппроксимации на дискретном наборе расчетных узлов (частиц), на которые разбивается моделируемая среда, со своими дискретными значениями функций [1].

Функция распределения $f(\boldsymbol{r})$ представляется через интегральный вид:

$$f(\boldsymbol{r}) = \int f(\boldsymbol{r}')\, \delta(\boldsymbol{r} - \boldsymbol{r}')\, d\boldsymbol{r}' \approx \int f(\boldsymbol{r}')\, W(\boldsymbol{r} - \boldsymbol{r}', h)\, d\boldsymbol{\Omega}, \qquad (1)$$

где:

f – произвольная непрерывная функция радиус-вектора,

δ – дельта-функция Дирака,

W – сглаживающее ядро (функция),

h – радиус сглаживания.

Для численной реализации интеграл заменяется суммой по частицам:

$$f(\boldsymbol{r}) \approx \sum_{j=1}^{N} m_j \frac{f_j}{\rho_j} W(\boldsymbol{r} - \boldsymbol{r}j, h), \qquad (2)$$

где:

m_j – масса частицы j,

ρ_j – плотность в позиции $\boldsymbol{r}_j$,

f_j – значение функции в этой точке.

Аналогично можно получить аппроксимацию по частицам градиента функции через градиент сглаживающего ядра:

$$\nabla f(\boldsymbol{r}) \approx -\sum_{j=1}^{N} m_j \frac{f_j}{\rho_j} \nabla W(\boldsymbol{r} - \boldsymbol{r}j, h), \qquad (3)$$

где градиент ∇W вычисляется относительно частицы j.

Для корректности и эффективности вычислительного метода на сглаживающее ядро W накладывают ряд ограничений:

1) Условие нормировки:

$$\int W(\boldsymbol{r} - \boldsymbol{r}', h)\, d\boldsymbol{\Omega} = 1,$$

2) Аппроксимация дельта-функции Дирака:

$$\lim_{h \to 0} W(\boldsymbol{r} - \boldsymbol{r}', h)\, d\boldsymbol{\Omega} = \delta(\boldsymbol{r} - \boldsymbol{r}'),$$

3) Компактность носителя:

$$W(\boldsymbol{r} - \boldsymbol{r}', h) = 0, \text{ для } |\boldsymbol{r} - \boldsymbol{r}'| > kh,$$

где k – масштабный множитель,

4) Неотрицательность:

$$W(\boldsymbol{r} - \boldsymbol{r}', h) \geq 0$$

5) Симметричность.

Более того, ядро должно быть как минимум дважды непрерывно дифференцируемым, чтобы обеспечить согласованную дискретизацию дифференциальных уравнений в частных производных второго порядка. Условие положительности не является строго обязательным, но в контексте физических симуляций ядра с отрицательными значениями могут привести к некорректным результатам, поэтому их следует избегать.

Основным выбором для сглаживающего ядра является кубический сплайн ядра, который удовлетворяет всем вышеуказанным свойствам (ограничениям):

$$W(\boldsymbol{r}, h) = \sigma_d \begin{cases} 6(q^3 - q^2) + 1, & 0 < q \leq \frac{1}{2} \\ 2(1-q)^2, & \frac{1}{2} < q \leq 1 \\ 0 & \end{cases}, \qquad (4)$$

где $q = \frac{||\boldsymbol{r}||}{h}$, а нормировочные коэффициенты для соответствующих размерностей d = 1, 2, 3 равны:

$\sigma_1 = \frac{4}{3h}$,

$\sigma_2 = \frac{40}{7\pi h^2}$,

$\sigma_3 = \frac{8}{\pi h^3}$. (5)

Для реализации метода важную роль играет радиус сглаживания h, влияющий на точность и эффективность вычисления. Если выбранный h будет слишком маленьким, то это приведет к снижению точности из-за недостаточного количества попадания частиц, а если слишком большим – детали полей в частице и их свойства будут теряться в процессе сглаживания.

Существует несколько методов при выборе радиуса сглаживания, такие как:

1) Глобальный радиус сглаживания: Одинаковый для всех частиц.
2) Индивидуальный радиус сглаживания: Индивидуальный для каждой частицы и зависит от начального распределения плотности.
3) Динамический радиус сглаживания:

   Выбор h в соответствии с текущей плотностью среды в частице i:

$$h_0 i = h \sqrt[D]{\frac{\rho_{0i}}{\rho_i}}, \qquad (6)$$

где:

h_0 – начальный радиус сглаживания,

ρ_0 – начальная плотность,

D – размерность пространства.

Для второго и третьего метода при расчете взаимодействий частиц i и j нужно, чтобы они оказывали симметричное влияние друг на друга, что следует из третьего закона Ньютона. Один из подходов заключается в использовании функции с симметричной относительно аргументов для радиуса сглаживания, таких как:

1. Арифметическое среднее: $h_{ij} = \frac{hi+hj}{2}$,

2. Геометрическое среднее: $h_{ij} = \frac{2\,hi\,hj}{hi+hj}$,
3. Минимальный радиус: $h_{ij} = min(h_i, h_j)$,
4. Максимальный радиус: $h_{ij} = max(h_i, h_j)$.

Несмотря на преимущества над другими методами, SPH имеет свои недостатки, указанные в [1] и [6], и к ним относят:

1. Повышенное требование к вычислительным ресурсам,
2. Недостаточная проработка вопросов сходимости и устойчивости,
3. Сложность задания граничных условий,
4. Трудность моделирования ударных волн,
5. Возможность проникновения частиц друг сквозь друга.

1.2. Применение SPH-метода в моделировании жидкостей

Метод сглаженных частиц широко применяется в моделировании жидкостей благодаря своей способности эффективно описывать сложные гидродинамические явления, такие как свободные поверхности, разрывы, турбулентность и взаимодействие жидкостей с твердыми телами. В отличие от традиционных сеточных методов, SPH не требует явного отслеживания границ раздела сред, гарантирует сохранение масс частиц без дополнительного вычисления, что делает его полезным для задач с большими деформациями и динамически изменяющими границами.

Ключевыми этапами моделирования жидкости методом SPH являются:

1) Дискретизация жидкости:

   Жидкость представляется в виде набора частиц, обладающая массой, плотностью, скоростью и давлением.

2) Вычисление плотности:

   Плотность вычисляется через суммирование вкладов от соседних частиц:

$$\rho_i = \sum_{j=1}^{N} m_j\, W(\boldsymbol{r_i} - \boldsymbol{r_j}, h) \qquad (7)$$

3) Расчет сил:

Давление:

$$p_i = k\,(\rho_i - \rho_0) \quad (8)$$

Сила вязкости:

$$f_{visco} = \mu\,\nabla^2\mathbf{v}, \quad (9)$$

$$\nabla^2\mathbf{v_i} = \sum_{j=1}^{N} \frac{m_j}{\rho_j}\boldsymbol{v_j}\nabla^2 W_{ij},$$

$$\nabla^2\mathbf{v_i} = -m_i \sum_{j=1}^{N} \frac{m_j}{\rho_j}\,\frac{r_{ij}\,\mathbf{v}_{ij}}{||r_{ij}||^2+0.01h^2}\,\nabla W_{ij}, \quad (10)$$

где μ – коэффициент вязкости,

$r_{ij} = r_i - r_j$,

$\mathbf{v}_{ij} = \mathbf{v}_i - \mathbf{v}_j$.

Сила давления:

$$f_{press} = -m_i \sum_{j=1}^{N} m_j\,(\frac{p_i}{\rho_j^2} + \frac{p_j}{\rho_j^2})\nabla W_{ij}, \quad (11)$$

4) Интегрирование уравнений движения: Обновление скоростей и позиций частиц.

Глава 2. Практическая часть

В данной главе представлены этапы практической реализации проекта: разработка моделей бокалов, моделирование наливания жидкости через SPH-метод и анализ результатов.

Для проведения симуляции была написана программная модель в Python с применением библиотек: NumPy для векторных вычислений, Matplotlib для визуализации, SciPy для интерполяции геометрии бокалов и Numba для ускорения вычислительных частей кода. Моделирование проводилось в трёхмерном пространстве с учетом основных физических сил: давления, вязкости и гравитации.

2.1. Разработка моделей бокалов

В рамках проекта были реализованы три основные геометрические модели бокалов в классе GlassShape:

1. Бокал с круглой чашей, известный как «Copa» (Round)
2. Бокал с сужающейся к верху чашей – «Бургундия» (Burgundy)
3. Бокал с конусообразной чашей – «Мартиника» (Martini)

Параметры чаши каждого бокала задается массивом высот z и соответствующие радиусы r. Для определения радиуса бокала на произвольной высоте использована интерполяция scipy.interpolate.interp1d. Ножка и основание выполнены идентично для всех бокалов. Код представлен в приложении №1.

2.2. Численное моделирование наливания жидкости

В основе численного моделирования жидкости лежит уравнение Навье-Стокса, описывающий движение вязкой жидкости:

$$\rho\frac{D\boldsymbol{v}}{Dt} = -\nabla p + \mu\nabla^2\boldsymbol{v} + f_{ext},$$

В SPH-методе уравнение реализуется через сумму силу давления (11), силу вязкости (9) и силу тяжести (См. приложение №2: функция sum_forces)

Для моделирования жидкости были заданы частицы со следующими параметрами:

mass – масса частицы,

rest_density – начальная плотность,

max_particles – максимальное количество частиц.

h – радиус сглаживания,

k – коэффициент жесткости давления,

mu – коэффициент вязкости,

dt – шаг по времени,

particles – массив для координат,

velocities – массив для скорости.

Также были заданы следующие функции:

Генерация новых частиц (add_particles): Частицы жидкости создаются в области, моделирующей горлышко бутылки (source_pos) с начальной скоростью направленной вниз по z (-5.0) и небольшим разбросом позиции и скорости. Интенсивность генерации – до 10 частиц за шаг по времени.

Расчет плотности (density): Плотность частицы *i* рассчитана согласно формуле (7).

Расчет сил (sum_forces): Сила давления рассчитана по формуле (11) и сила вязкости по формуле (10).

Взаимодействие частиц со стенками бокала (collide): При столкновении корректируется позиция частицы, отражается нормальная составляющая скорости с коэффициентом реституции 0.6 и добавляется небольшой случайный шум (scale).

Интегрирование (update): Обновление скорости и положения частиц через явную схему Эйлера первого порядка.

Код представлен в приложении №2.

2.3. Анализ результатов

Для оценки эффективности формы бокала в плане минимизации брызг при наливании жидкости была использована следующая формула (См. приложение №2: функция count):

$$\text{Эффективность (\%)} = \frac{\text{Количество внутри бокала}}{\text{Общее количество частиц}} * 100\%$$

Функция count определяет частицу как находящуюся внутри бокала, если:

1. Её координата по оси z находится между текущим уровнем жидкости (fill_height) и высотой (z_center * 0.4);
2. Её расстояние от оси бокала в плоскости xy (r_xy = sqrt(x^2 + y^2)) меньше или равно максимально допустимому радиусу бокала на данной высоте, с учетом масштабирования модели.

Статистика эффективности выводилась каждые 10 кадров анимации из 60 (См. приложение №3).

Ниже представлены кадры анимации №1, 30 и 60 процесса наливания жидкости в бокалы (Рис. 1-3):

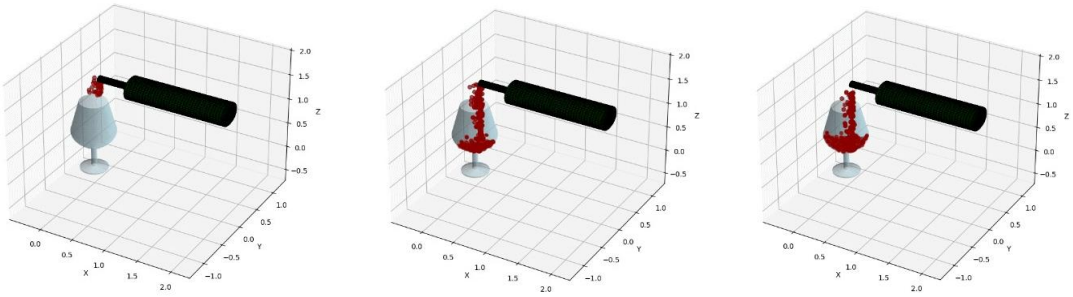

Рис. 1. Кадры симуляции в бокале Бургундия

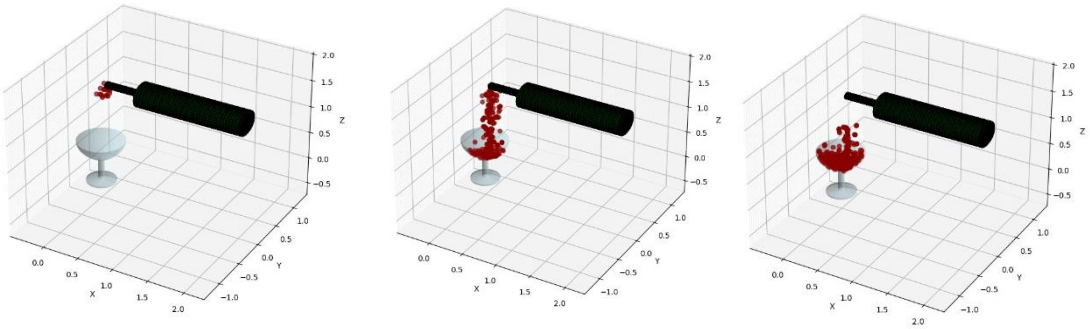

Рис. 2. Кадры симуляции в бокале Сора

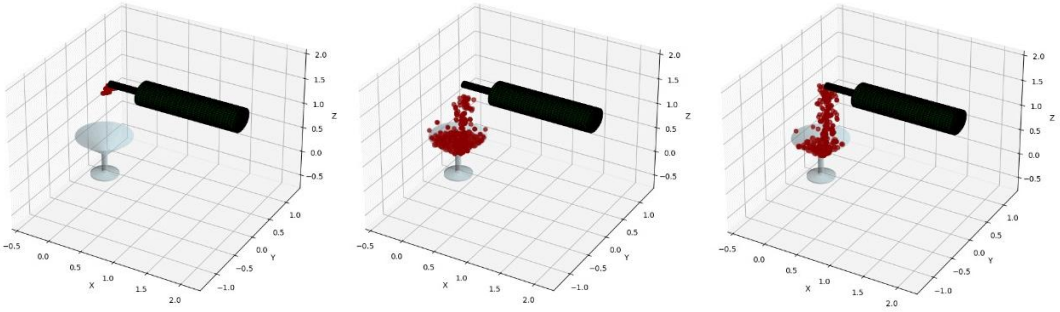

Рис. 3. Кадры симуляции в бокале Мартиника

Конечные результаты представлены ниже в виде таблицы:

Модель бокала	Количество частиц	Эффективность (%)
Бургундия	479/500	95,8%
Сора	370/500	74,0%
Мартиника	328/500	65,6%

Табл. 1

Анализ результатов:

Форма с плавным сужением к верху (Бургундия) демонстрирует наилучшие результаты, минимизируя брызги.

Круглая форма (Сора) демонстрирует хорошую, но менее стабильную эффективность, так как при кадре №50 достигается максимальная эффективность 75,4%, но в конце наблюдается снижение до 74,0% из-за отсутствия сужения, приводящая к отскоку частиц.

Конусообразная форма (Мартиника) показала самую низкую и нестабильную эффективность по сравнению с предыдущими. После достижения пика (76.8%) снижается до 65,6%, это объясняется тем, что форма стенок бокала приводит к сильным отскокам частиц и их вылету при заполнении.

Вывод:

Оптимальная форма – Бургундия. Её сужающая форма наиболее эффективно снижает брызги -–95,8% удержания жидкости.

Наихудшая форма – Мартиника. Конусообразная форма не подходит для снижения брызг – 65,8% удержания жидкости.

Заключение

В рамках курсового проекта была проведена оптимизация формы бокала с использованием SPH-метода для минимизации брызг при наливании жидкости.

В процессе работы над этим проектом была изучена теоретическая основа SPH-метода, а также его основные достоинства и недостатки, разработан программный комплекс, реализующий симуляцию наливания и поведения жидкости, были проанализированы результаты моделирования.

Таким образом, поставленные цель и задачи курсового проекта были достигнуты. Анализ форм показал, что форма бокала существенно влияет на количество брызг. Подтверждена эффективность формы с плавно сужающейся верхней частью чаши (Бургундия).

Для дальнейших расследований можно рассмотреть дополнительные параметры, такие как скорость и угол наливания, более другие формы бокала и выбрать другой радиус сглаживания.

Список литературы

1. И. В. Абрамов, М. А. Алексеев, Д.О. Левченко, С. А. Моргунов Метод гидродинамики сглаженных частиц // Известия Московского государственного индустриального университета. – 2011 – №4. – С. 2 – 11.
2. Liu G. R., Liu M. B. Smoothed particle hydrodynamics, a meshfree particle method. // World Scientific Publishing Co. Pte. Ltd, 2003. — P. 473.
3. Monaghan J.J. Smoothed particle hydrodynamics // Annu. Rev. Astron. Astrophys. — 1992. — 30: 543 — 574.
4. Springel V. The cosmological simulation code GADGET-2, Mon. Not. R. Astron. Soc. 364. — P. 1105 — 1134.
5. Dan Koschier, Jan Bender, Barbara Solenthaler, Matthias Teschner Smoothed Patricle Hydrodynamics Techniques for the Physics Based Simulation of Fluids and Solids // EUROGRAPHICS. – 2019.
6. О. П. Стояновская, В. Н. Снытников Особенности SPH-метода решения газодинамических уравнений при моделировании нелинейных волн в двухфазной гравитирующей среде // Математическое моделированиею – 2010 – том 22, №5. – С. 29-44.

Приложения 1

class_for_3d_models.py

```
import numpy as np
from scipy.interpolate import interp1d


class GlassShape:
    def __init__(self):
        self.z = None
        self.r = None
        self.z_center = None
        self.r_interp_func = None
        self.fill_height = 0.1 * 0.4

    def create_shape(self):
        return ("Понадобится")

    def get_interpolator(self):
        self.r_interp_func   =   interp1d(self.z,   self.r,   bounds_error=False,
fill_value="extrapolate")

    def get_glass_params(self):
        return {
            'z': self.z,
            'r': self.r,
            'z_center': self.z_center,
            'r_interp_func': self.r_interp_func,
            'fill_height': self.fill_height
        }
    def get_name(selfself):
        return ("Понадобится")


class Round(GlassShape):
    def create_shape(self):
        self.z = np.linspace(-1, 1, 150)
        self.r = 0.8 * (1 - (self.z - 1) ** 2)
        max_idx = np.argmax(self.r)
        self.z_center = self.z[max_idx]

        self.r[self.z <= 0.1] = 0.1
        self.r[self.z < -0.9] = 0.5
```

```
        self.get_interpolator()

    def get_name(selfself):
        return ("Copa")


class Burgundy(GlassShape):
    def create_shape(self):
        self.z = np.linspace(-1, 2.5, 150)
        self.r = np.zeros_like(self.z)
        r_parabola = 0.8 * (1 - (self.z - 1) ** 2)
        max_idx = np.argmax(r_parabola)
        self.z_center = self.z[max_idx]
        r_center = r_parabola[max_idx]
        self.r[self.z <= self.z_center] = r_parabola[self.z <= self.z_center]
        self.r[self.z > self.z_center] = np.linspace(r_center, 0.5, np.sum(self.z
> self.z_center))

        self.r[self.z <= 0.1] = 0.1
        self.r[self.z < -0.9] = 0.5
        self.get_interpolator()

    def get_name(selfself):
        return ("Бургундия")


class Martini(GlassShape):
    def create_shape(self):
        self.z = np.linspace(-1, 1, 150)
        v_start = 0.0
        v_end = 1
        v_radius_top = 1.0
        self.z_center = v_end
        self.r = np.zeros_like(self.z)
        v_mask = (self.z >= v_start) & (self.z <= v_end)
        self.r[v_mask] = (self.z[v_mask] - v_start) / (v_end - v_start) *
v_radius_top
        self.r[self.z <= 0.1] = 0.1
        self.r[self.z < -0.9] = 0.5
        self.get_interpolator()

    def get_name(selfself):
        return ("Мартиника")
```

```
def create(choice):
    glasses = {
        1: Round,
        2: Burgundy,
        3: Martini
    }
    if choice not in glasses:
        raise ValueError("Invalid bottle choice")

    glass = glasses[choice]()
    glass.create_shape()
    return glass
```

Приложение 2

main.py

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib.animation import FuncAnimation
from numba import njit
from datetime import datetime
from class_for_3d_models import create

print("Выберите форму бокала:")
print("1: Круглая чаша (Copa)")
print("2: Круглая чаша сужающая сверху (Бургундия)")
print("3: Конусообразная (Мартиника)")

glass_choice = int(input("Введите номер: "))
glass = create(glass_choice)
glass_params = glass.get_glass_params()

z = glass_params['z']
r = glass_params['r']
z_center = glass_params['z_center']
r_interp_func = glass_params['r_interp_func']
fill_height = glass_params['fill_height']

theta = np.linspace(0, 2 * np.pi, 100)
Z, Theta = np.meshgrid(z, theta)
R = np.tile(r, (len(theta), 1))

X = 0.4 * R * np.cos(Theta)
Y = 0.4 * R * np.sin(Theta)
Z = 0.4 * np.tile(z, (len(theta), 1))

min_z_glass = np.min(Z)

z_bottle = np.linspace(1, 3, 100)
r_bottle = -0.15 * np.ones_like(z_bottle)
r_bottle[(z_bottle > 1) & (z_bottle < 2.5)] = 0.2
r_bottle[z_bottle > 2.5] = 0.05
Zb, thetab = np.meshgrid(z_bottle, theta)
Rb = np.tile(r_bottle, (len(theta), 1))
Xb = Rb * np.cos(thetab)
```

```
Yb = Rb * np.sin(thetab)
Zb = np.tile(z_bottle, (len(theta), 1))

Xb_rot = 3.1 - Zb
Yb_rot = Yb
Zb_rot = -Xb + 1.5

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, color='lightblue', alpha=0.5)
ax.plot_surface(Xb_rot,  Yb_rot,  Zb_rot,  color='green',  edgecolor='k',
alpha=0.5)
scat = ax.scatter([], [], [], color='darkred', s=30, alpha=0.7)
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")

h = 0.1
mass = 0.02
rest_density = 500
k = 3.0
mu = 0.1
g = np.array([0, 0, 9.81])
max_particles = 500
particles = np.full((max_particles, 3), np.nan)
velocities = np.zeros((max_particles, 3))
source_pos = np.array([0.1, 0.0, 1.5])
total_particles_added = 0
dt = 0.015


@njit
def W(r, h):
    q = np.linalg.norm(r) / h
    sigma = 8 / (np.pi * h ** 3)
    if 0 <= q <= 1 / 2:
        return sigma * (6 * (q ** 3 - q ** 2) + 1)
    elif 1 / 2 < q <= 1:
        return sigma * (2 * (1 - q) ** 3)
    else:
        return 0
```

```
@njit
def grad_W(r, h):
    q = np.linalg.norm(r) / h
    sigma = 8 / (np.pi * h ** 3)
    if q == 0:
        return np.zeros_like(r)
    if 0 <= q <= 1 / 2:
        return sigma * (18 * q - 12) * r / (h ** 2 * q)
    elif 1 / 2 < q <= 1:
        return sigma * (-6 * (1 - q) ** 2) * r / (h ** 2 * q)
    else:
        return np.zeros_like(r)


@njit
def density(particles):
    N = len(particles)
    rho = np.zeros(N)
    for i in range(N):
        if np.isnan(particles[i, 0]):
            continue
        for j in range(N):
            if np.isnan(particles[j, 0]):
                continue
            r_vec = particles[i] - particles[j]
            rho[i] += mass * W(r_vec, h)
    return rho


@njit
def sum_forces(particles, velocities, rho):
    N = len(particles)
    forces = np.zeros_like(particles)
    pressure = k * (rho - rest_density)
    for i in range(N):
        if np.isnan(particles[i, 0]):
            continue
        for j in range(N):
            if i == j or np.isnan(particles[j, 0]):
                continue
            r_vec = particles[i] - particles[j]
            dist = np.linalg.norm(r_vec)
            if dist > h or dist == 0:
```

```
                    continue
                f_pressure = -mass * (pressure[i] / rho[i] ** 2 + pressure[j] /
rho[j] ** 2) * grad_W(r_vec, h)
                f_viscosity = mu * 10 * (mass / rho[j]) * (((velocities[i] -
velocities[j]) * (particles[i] - particles[j]))/(dist ** 2 + 0.01 * h ** 2)) *
grad_W(r_vec, h)
                forces[i] += f_pressure + f_viscosity
            forces[i] += g * mass
        return forces


    def add_particles():
        global total_particles_added
        num_new = 10
        added = 0
        for i in range(max_particles):
            if np.isnan(particles[i, 0]):
                angle = np.random.rand() * 2 * np.pi
                radius = np.random.rand() * 0.15
                offset = np.array([radius * np.cos(angle), radius *
np.sin(angle), 0.05 * (np.random.rand() - 0.5)])
                particles[i] = source_pos + offset
                velocities[i] = np.array([0.0, 0.0, -5.0]) + 0.3 *
(np.random.rand(3) - 0.5)
                added += 1
                if added >= num_new:
                    break
        total_particles_added += added


    def collide(particles, velocities):
        min_z_glass = np.min(z) * 0.4

        for i in range(len(particles)):
            if np.isnan(particles[i, 0]):
                continue
            z_pos = particles[i, 2]
            x_pos = particles[i, 0]
            y_pos = particles[i, 1]
            r_xy = np.sqrt(x_pos ** 2 + y_pos ** 2)

            z_original = z_pos / 0.4
            r_max = r_interp_func(z_original) * 0.4
```

```
        if r_xy > r_max:
            normal = np.array([x_pos, y_pos, 0]) / (r_xy + 1e-8)
            scale = r_max / r_xy * 0.99
            particles[i, 0] = x_pos * scale
            particles[i, 1] = y_pos * scale
            v_normal = np.dot(velocities[i], normal) * normal
            velocities[i] -= 1.3 * v_normal

        if z_pos <= fill_height:
            particles[i, 2] = fill_height + 0.001
            velocities[i, 2] = max(-0.1 * velocities[i, 2], 0)
            velocities[i, 0] *= 0.5
            velocities[i, 1] *= 0.5

        if z_pos < min_z_glass:
            particles[i, 2] = min_z_glass + 0.001
            velocities[i, 2] = max(-0.1 * velocities[i, 2], 0)
            velocities[i, 0] *= 0.5
            velocities[i, 1] *= 0.5


def count(particles):
    count = 0
    min_z_glass = np.min(z) * 0.4
    for i in range(len(particles)):
        if np.isnan(particles[i, 0]):
            continue
        z_pos = particles[i, 2]
        x = particles[i, 0]
        y = particles[i, 1]
        r_xy = np.sqrt(x ** 2 + y ** 2)

        z_original = z_pos / 0.4
        r_max = r_interp_func(z_original) * 0.4

        if min_z_glass <= z_pos <= z_center * 0.4 and r_xy <= r_max:
            count += 1
    return count


def set_axes_equal(ax):
    extents = np.array([getattr(ax, f'get_{dim}lim')() for dim in 'xyz'])
```

```
        centers = np.mean(extents, axis=1)
        maxsize = max(extents[:, 1] - extents[:, 0])
        for ctr, dim in zip(centers, 'xyz'):
            getattr(ax, f'set_{dim}lim')(ctr - maxsize / 2, ctr + maxsize / 2)


    set_axes_equal(ax)

    def update(frame):
        global particles, velocities, fill_height
        add_particles()
        valid = ~np.isnan(particles[:, 0])
        if np.count_nonzero(valid) > 0:
            rho = density(particles[valid])
            forces = sum_forces(particles[valid], velocities[valid], rho)
            accelerations = forces / rho[:, None]
            velocities[valid] += accelerations * dt
            particles[valid] += velocities[valid] * dt
            collide(particles, velocities)

        if total_particles_added < max_particles:
            fill_height += 0.001

        scat._offsets3d = (particles[:, 0], particles[:, 1], particles[:, 2])

        if frame % 10 == 0 and total_particles_added > 0:
            in_glass = count(particles)
            print(
                f"Кадр №{frame}: {in_glass}/{total_particles_added} частиц в
бокале ({100 * in_glass / total_particles_added:.1f}%)")

        return scat,


    ani = FuncAnimation(fig, update, frames=60, interval=30, blit=False)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"{glass.get_name()}_{timestamp}.gif"
    print(f"Сохранение анимации в файл: {filename}")
    ani.save(filename, writer='pillow', fps=30, dpi=100)
    plt.tight_layout()
    plt.show()
```

Приложение 3

Бокал «Бургундия»:

Кадр	Количество частиц	Эффективность (%)
0	0/10	0,0%
10	0/120	0,0%
20	81/220	36,8%
30	178/320	55,6%
40	273/420	66,4%
50	380/500	76,0%
60	479/500	95,8%

Табл. 2

Бокал «Сора»:

Кадр	Количество частиц	Эффективность (%)
0	0/10	0,0%
10	0/120	0,0%
20	76/220	34,5%
30	180/320	56,2%
40	281/420	66,9%
50	377/500	75,4%
60	370/500	74,0%

Табл. 3

Бокал «Мартиника»:

Кадр	Количество частиц	Эффективность (%)
0	0/10	0,0%
10	0/120	0,0%
20	80/220	36,4%
30	179/320	55,9%
40	280/420	66,7%
50	384/500	76,8%
60	328/500	65,6%

Табл. 4